Hephaestus: an in-browser open-source engine and databases for the thermodynamics and phase diagrams of slags, molten salts, and alloys

Michael E. Bustamante

Odinzen LLC, Houston, TX, United States

Corresponding author: michaelbusta@odinzen.io

ORCID: 0009-0009-9001-8151

Abstract

The Modified Quasichemical Model in the Quadruplet Approximation (MQMQA) is the standard thermodynamic description of short-range-ordered ionic melts, and it underpins the commercial databases used throughout pyrometallurgy. Open implementations exist inside larger frameworks, but no freely licensed oxide slag database has been available, so users of open tools have had the model without parameters to run through it. Hephaestus addresses the model and the data together. The engine is a small dependency-free C core with a Python interface and a WebAssembly build, so the same code runs embedded, scripted, or in a browser with nothing installed. It reads ChemSage-format files directly, covering MQMQA liquids, compound energy formalism solid solutions, and stoichiometric compounds. It reads the Thermo-Calc TDB dialect for alloys, including the Inden magnetic contribution real steel files require. It also defines uTDB, an openly specified extension carrying the MQMQA statements inside TDB grammar, demonstrated by shipped aluminum-recycling (Al-Zn alloy with its NaCl-KCl-$MgCl_2$ salt flux) and steelmaking (Fe-C steel with its FeO-MgO-SiO_2 slag) databases. One zero-install page calculates across all three containers. It calculates phase equilibria up to full ternary isothermal sections and liquidus projections, and Scheil-Gulliver solidification paths on any binary join. A NASA-polynomial ideal-gas engine, validated against Cantera, brings the gas equilibria a furnace atmosphere demands into the same browser; coupling the gas to a condensed melt is in progress. Every energy path is validated to machine precision against pycalphad, and the magnetic model additionally against the measured pure-iron transition temperatures. The companion databases cover the CaO-SiO_2, MgO-SiO_2, and FeO-SiO_2 liquids and the FeO-MgO-SiO_2 ternary with olivine and orthopyroxene solid solutions, together with an open chloride molten-salt family from LiCl-KCl to the NaCl-KCl-$MgCl_2$ ternary, assembled solely from published measurements with per-value provenance and documented limits. The code is MIT licensed and the databases are CC BY 4.0.

Keywords: CALPHAD; molten slag; molten salt; modified quasichemical model; thermodynamic database; phase diagram; gas equilibrium; WebAssembly; open data

(1) Overview

Introduction

Molten oxide slags and molten salts are dominated by short-range ordering, which the Modified Quasichemical Model in the Quadruplet Approximation describes through the distribution of cation-anion quadruplets [1]. The quadruplet formalism is the current form of the modified quasichemical family, which grew from the pair-approximation treatment of binary liquids [2] and its application to silicate slags [3]. The model is published in full, and it is the basis of the oxide and salt databases in FactSage, the software that carries most industrial slag work [4]; databases of this kind are embedded in everyday steelmaking practice, from slag design through virtual process simulation [5,6]. These processes are never purely condensed. The furnace atmosphere governs them, because the carbon monoxide to carbon dioxide ratio, and the oxygen partial pressure it fixes, decide whether a metal is reduced or a slag oxidized, so the gas belongs in the same calculation. That gas data lives in yet another open family, the NASA polynomials of combustion and aerospace. What separates a published model from a usable calculation is an implementation and a parameter set. On the implementation side the situation has improved. OpenCalphad brought a free general equilibrium code [7]; pycalphad, itself published in this journal [8], gained an MQMQA model with uncertainty quantification [9] and a companion parameter-fitting framework [10]; Thermochimica calculates MQMQA molten salts inside a nuclear fuel framework [11]. Both MQMQA implementations live inside larger frameworks with correspondingly large dependency stacks. On the data side, open infrastructure has been slower to arrive. The community has argued for years that phase-based data need open infrastructure [12], yet there is no free MQMQA oxide slag database. A student, a startup, or a plant engineer who wants to calculate a slag equilibrium with open tools has the model, possibly an implementation, and no parameters to run through it.

The workflows behind these tools also shape who can use them. FactSage [4] and Thermo-Calc [13] are mature commercial suites whose module interfaces and expertly curated databases serve licensed users well, and other commercial packages follow the same shape. OpenCalphad [7] and pycalphad [8] are free engines for the code-comfortable: command files or Python sessions over TDB databases, with no graphical calculator attached. CemGEMS [14] showed a third way for cement chemistry, a free web front end whose users install nothing, though its calculations still run on a project server; the GEMS ecosystem behind it runs its own container formats entirely, so the fragmentation extends beyond CALPHAD's two dialects. What none of them offers is a calculator that is zero-install, fully client-side, and open-data at once: a page anyone can open, load a database in either of the field's file dialects, and calculate against, with nothing transmitted anywhere. Hephaestus takes that seat deliberately as a complement rather than a competitor. It does not chase FactSage's breadth or Thermo-Calc's alloy depth; it is the low-barrier door into the same published models, with open parameters behind it and a fitting loop short enough that refitting an excess term against new measurements is an interactive step.

The container formats themselves deserve a moment, because they all serve one need, assessed Gibbs-energy functions behind a calculation, yet none reads another (Table 1). The community has begun to move on this: XTDB [15], from the OpenCalphad and NIST circle, proposes a unified successor in XML, the tagged text markup web pages are written in, and its schema already includes the MQMQA model. Its authors weighed keeping the compact human-readable

TDB notation against a schema with built-in verification, and chose XML because “even small variations of the TDB format will create problems” [15]: they are engineering for the future toolchain. This work approaches the same goal from the installed base instead, reading both legacy dialects as they are and publishing a minimal in-place extension, uTDB, for the one model TDB lacks; the two roads are wrappers around one destination, and converting between them is mechanical. Read as a path, the formats run TDB to uTDB to XTDB, the familiar files extended in place today and translated into the XML successor once its tooling arrives.

Table 1: The schools of thermodynamic database containers. One need, assessed Gibbs energies behind a calculation, split across mutually unreadable families; the present work reads the two CALPHAD dialects and their unified extension in one engine.

School	Native containers	Carried by
CALPHAD, ChemSage lineage	ChemSage .dat	FactSage, ChemApp, Thermochimica; the only legacy carrier of MQMQA melts
CALPHAD, Thermo-Calc lineage	TDB	Thermo-Calc, OpenCalphad, pycalphad; alloy CEF
CALPHAD, XML successor	XTDB [15]	proposed unified schema, MQMQA included
Geochemical / aqueous	GEMS containers, PhreeqC data files	GEM-Selektor behind CemGEMS [14]; speciation models
Combustion / gas phase	NASA-polynomial files (CHEMKIN, Cantera)	ideal-gas and pure condensed species, no solution models
First-principles era	JSON records served over the web	calculated formation energies and finite-temperature properties (the Materials Project [16] and kin)

Hephaestus is a deliberate response to that gap, and it treats the engine and the data as one contribution because neither is useful alone. The engine is small on purpose. Its core is plain C with no dependencies, which makes it easy to embed and lets the identical code compile to WebAssembly, so the full calculator runs in a web browser with nothing installed and no data leaving the user’s machine. The database is open on purpose. Every parameter in it derives from published measurements, each value carries its provenance down to table and page, and the known shortcomings are documented next to the numbers they affect. The present work describes both, their validation against pycalphad and against measured phase equilibria, and the ways they can be reused. None of it asks the reader to program. For the experimentalist who never writes code, the browser application is the entire product: a page to open, a database to click, and a diagram to read. The sections that follow describe the machinery underneath for those who want it. Supplementary Material S1 is a plain-language primer for the non-

programmer, starting at what a database file is and ending at a first calculation and how to read it. Its closing section teaches what an assessment is built from, so a reader who wants to contribute data for open publication, or to commission a file, arrives knowing what is needed.

Implementation and architecture

Hephaestus is layered (Figure 1). A C core holds all thermodynamics. A thin Python package calls the C library directly through cffi, so importing it requires a prebuilt shared library and no compiler. An emscripten build compiles the same C into a single WebAssembly file behind a browser application. Nothing in the chain duplicates a formula, so a fix in the core reaches every layer.

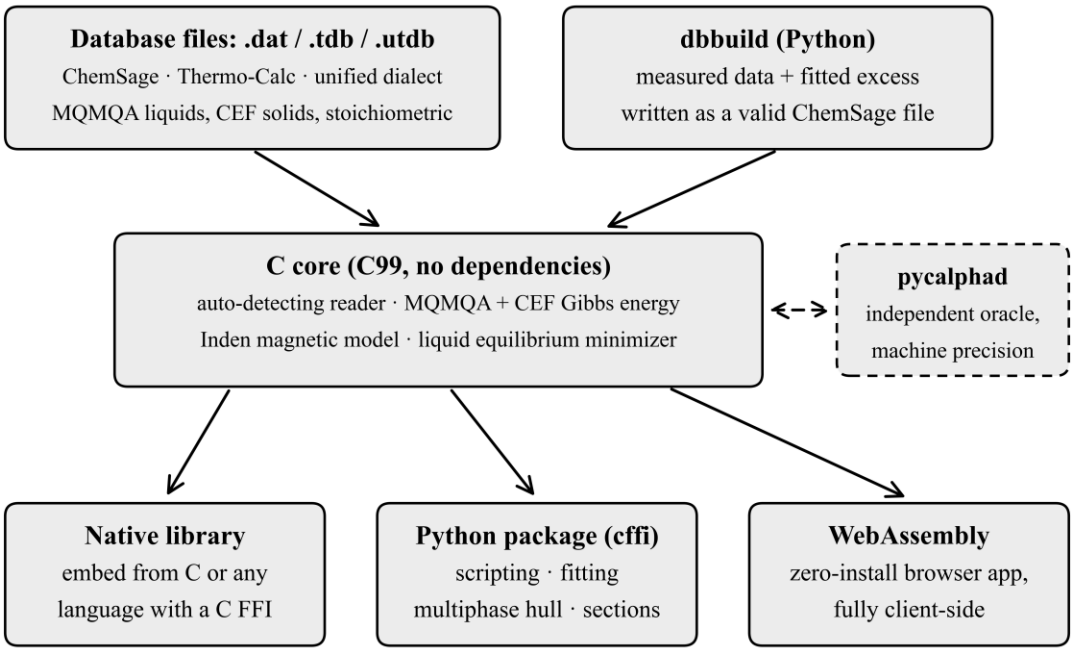

Figure 1: Architecture of Hephaestus. A single dependency-free C core reads the field's three database dialects (ChemSage .dat, Thermo-Calc .tdb with the Inden magnetic model, and the unified .utdb) and calculates all thermodynamics; the same core is used as a native library, from Python through cffi, and compiled to WebAssembly behind the browser application. dbbuild writes new ChemSage files from measured data, and every energy path is validated against pycalphad.

The core implements the MQMQA Gibbs energy of Pelton, Chartrand and Eriksson [1]: the pair reference energy, the configurational entropy of the quadruplet distribution, and the excess energy with composition-dependent interaction terms, together with the recursive coordination-number rules, including reciprocal quadruplets. Solid solutions use the compound energy formalism (CEF) [17], with sublattice site fractions, per-sublattice ideal entropy, and Redlich-Kister excess terms; the Gibbs energy is normalized per mole of real atoms, so vacancy-bearing phases come out on the same basis pycalphad uses. A reader for the ChemSage data format, the file lineage of the ChemSage and ChemApp equilibrium programs [18], loads MQMQA liquids

(SUBQ and SUBG blocks), compound energy formalism phases (SUBL, including the magnetic variant), and stoichiometric compounds, with multi-interval Gibbs functions and the additional-term forms such as $T^{0.5}$ and ln T. The format has no public specification, so the reader's grammar was written against the format as handled by pycalphad's open-source parser (the piece of code that turns a database text file into numbers an engine can use) and proven by loading pycalphad's own test databases.

Equilibrium is calculated at two levels. The core minimizes the liquid Gibbs energy over quadruplet fractions under element balance. The Python layer adds an exact one-dimensional solve for common-anion binary liquids, where mass balance pins the quadruplet distribution to a single degree of freedom, and a multiphase construction by dense sampling and the lower convex hull, the same global method pycalphad uses. For a binary the hull yields tie-lines with a test that separates true two-phase edges from adjacent samples of one continuous curve. For a ternary the hull runs over the cation simplex, and each facet is a tie-triangle or tie-line read directly with the lever rule. On top of this sit isothermal sections and liquidus projections for the full FeO-MgO-SiO_2 system.

The same core also reads the Thermo-Calc TDB (thermodynamic database) dialect, the lingua franca of alloy CALPHAD shared by Thermo-Calc [13], OpenCalphad [7], and pycalphad [8]. TDB files parse into the same internal database, so the accessors, the Python layer, and the browser work unchanged, and the format is auto-detected on load. The supported subset covers elements, species, piecewise FUNCTION expressions with nested references, compound energy formalism phases with vacancies, binary Redlich-Kister interactions of any order (ternary interaction parameters are outside the subset and rejected with the stated reason), and the Inden magnetic contribution (TC and BMAGN parameters with their antiferromagnetic factors), which brings real steel files into scope; models outside the subset, such as order-disorder partitioning and the ionic two-sublattice liquid, are rejected with a stated reason rather than calculated incompletely. Temperature extrapolation above a function's last interval follows the Thermo-Calc convention for TDB data while ChemSage data keep their own zero-above convention. MQMQA liquids have no TDB representation, so melts remain the province of the .dat dialect; the two families meet in one engine.

From Python the calculation chain is three calls, identical for either dialect:

```
from mqmqa import Database
from mqmqa.equilibrium import build_inputs, c_equilibrate

db  = Database.read("FeO-MgO-SiO2-combined.dat")   # or any .tdb
inp = build_inputs(db, db.phase_index("FEO-MGO-SIO2-LIQUID"), 1873.0)
res = c_equilibrate(inp, {"FE": 0.2, "MG": 0.2, "SI": 0.2, "O": 1.4})
# res["GM"], res["X"]: molar Gibbs energy and the quadruplet distribution
```

Read as plain steps: the two import lines fetch the named tools from the installed package. Database.read opens the file, detects its dialect, and loads every phase and parameter into memory. build_inputs gathers what the solver needs about one named phase at one temperature, here the slag liquid at 1873 K. c_equilibrate then finds the internal state of lowest Gibbs energy for the given amounts of each element and returns a small table: res["GM"] is the molar Gibbs energy and res["X"] is the melt's internal structure, the quadruplet fractions. Every calculation in this paper is some loop around these three calls.

The Python package also contains dbbuild, a declarative route from measured data to a loadable database. A user describes components with sourced endmember thermodynamics and fitted binary excess terms, and dbbuild writes a valid ChemSage file; a fitting routine obtains the excess parameters from measured component activities using the engine itself as the forward model. The builder covers single-anion oxide systems of two to four components, and multicomponent liquids are assembled from their binaries in the usual way.

The browser application, hosted at https://odinzen.github.io/hephaestus-mqmqa-public/, exposes seven tools. A calculator works over any loaded database (ChemSage .dat or Thermo-Calc .tdb, auto-detected, with up to three uploaded files held per session). A binary-join phase-diagram tool offers tap-to-read point comparison, and a Scheil-Gulliver solidification panel on the same join draws the equilibrium lever-rule path alongside. A viewer shows the assessed FeO-MgO-SiO_2 diagrams, and a live isothermal-section solver calculates the full phase assemblage at a chosen temperature in a second or two, for the shipped ternaries or for any loaded three-cation file (Figure 2). An interactive eutectic builder pre-fills its component names, melting points, and fusion enthalpies from the loaded database. A gas-equilibrium calculator solves ideal-gas equilibrium over an open NASA-polynomial database, its own engine compiled into the same WebAssembly core. Loading an alloy TDB reshapes the workspace: the elements become the end members and every compound energy formalism phase spanning the join enters the same convex hull construction. Everything runs client side. The page holds a strict content security policy, escapes all file-derived strings, makes no third-party requests, and never transmits a loaded file, which matters to industrial users whose compositions are confidential.

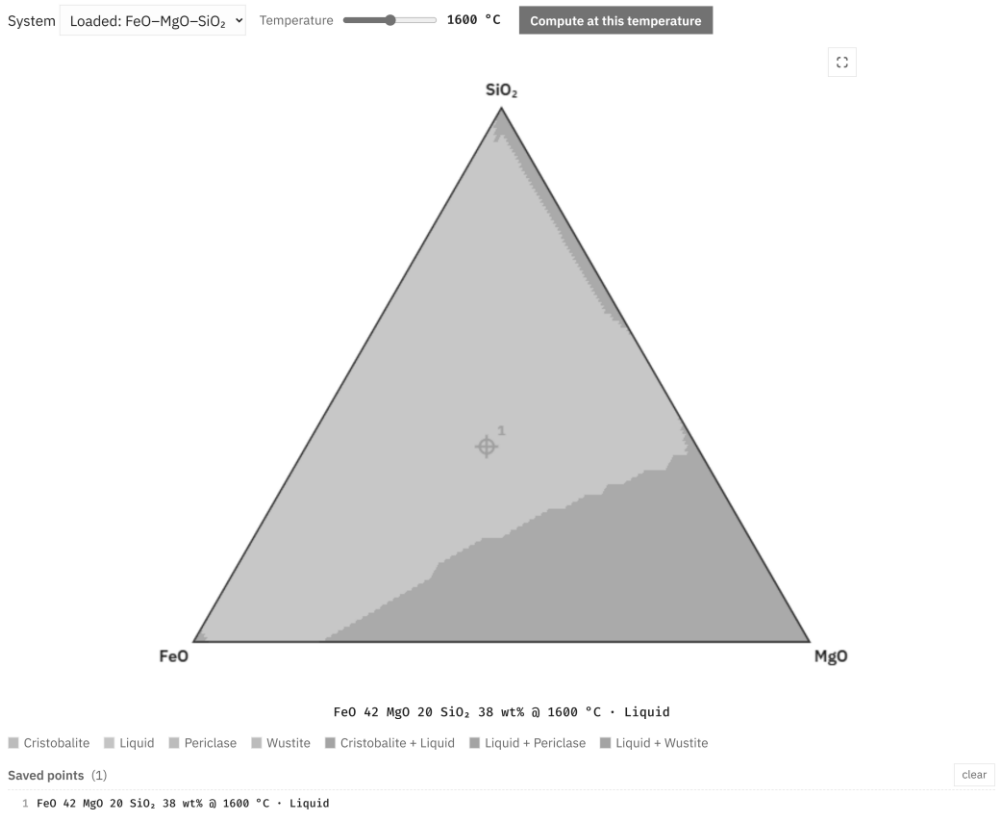

Figure 2: The live solver in the browser application (light theme). The user loads the shipped database, picks a temperature, and presses Compute; the WebAssembly engine then solves the full FeO-MgO-SiO_2 phase assemblage client side (here 861 composition samples and 595 tie-facets in about two seconds). Tapping a composition reads its stable phases and pins it to a comparison list; hovering previews on desktop.

The database is the second half of the contribution. Endmember oxide thermodynamics come from open compilations, chiefly Robie and Hemingway [19] and the NIST-JANAF tables [20]. Liquid excess parameters are fitted to published measurements: Knudsen-effusion and gas-slag equilibrium activities for CaO-SiO_2 [21,22], the measured iron-saturated activities collected by Björkman for FeO-SiO_2 [23], and the melting and immiscibility constraints of the classical phase diagram studies [24-26]. Solution-calorimetric compound enthalpies [27] anchor the solids. Where published calorimetry cannot constrain a liquid, melt-mixing enthalpies from machine-learned interatomic potentials [28,29] serve as auxiliary anchors after bias correction against measured formation enthalpies; each such use is recorded in the provenance files. First-principles codes sit upstream of this pipeline. A density functional theory (DFT) engine such as Quantum ESPRESSO [30] or VASP [31] calculates the energy of one configuration at a time, at zero kelvin unless phonon or molecular-dynamics machinery is layered on top. It supplies candidate data for an assessment to weigh, not assessed multicomponent Gibbs functions, and repositories such as the Materials Project [16] serve such results at scale. That physics enters the present databases only through potentials trained on large DFT corpora. MatterSim [28] and Orb [29] supply the melt anchors above, and a further graph-network potential, SevenNet [32], was used to corroborate the olivine mixing enthalpy against the calorimetric record [33] through configuration-averaged relaxed supercells, with endmember differences cancelling any constant

offset; the check is recorded in the repository and changed no fitted parameter. The olivine and orthopyroxene solid solutions take their endmembers from Robie and Hemingway [19] and their mixing parameters from solution calorimetry [33]. The assembled ternary reproduces the topology of the measured MgO-FeO-SiO_2 liquidus surface [24], with the olivine field quantitative and the known offsets stated. No parameter is taken from any prior assessment or commercial database; published assessments of these systems [34] are used only as method literature. The same engine and file format also serve molten salts: the repository ships an open chloride family alongside the slags (LiCl-KCl, NaCl-KCl with its continuous halite solid solution, KCl-$MgCl_2$ with the double salt $KMgCl_3$, NaCl-$MgCl_2$, and the assembled NaCl-KCl-$MgCl_2$ ternary), built the same way from evaluated endmember tables [35], measured mixing calorimetry [36], and the measured eutectic, liquidus, and solvus record of the halide literature [37], and loadable in the browser application with one click. The nuclear community's MSTDB-TC [38] covers molten salts for reactor work under a registration license; the chloride family here is complementary, small, and CC BY. Each system folder carries a provenance file that names every source, every modeling judgment, and every known limit. The largest limit is stated plainly: the CaO-SiO_2 and MgO-SiO_2 silica-rich miscibility gaps sit at the right compositions but too high in temperature, a documented consequence of the single-common-anion excess form. Figure 3 shows the assembled ternary as the engine calculates it from the shipped database files, in the two views the application offers; Table 2 summarizes each fitted liquid, the data behind it, and what it reproduces. The code is MIT licensed at the repository root and the database directory carries its own CC BY 4.0 license.

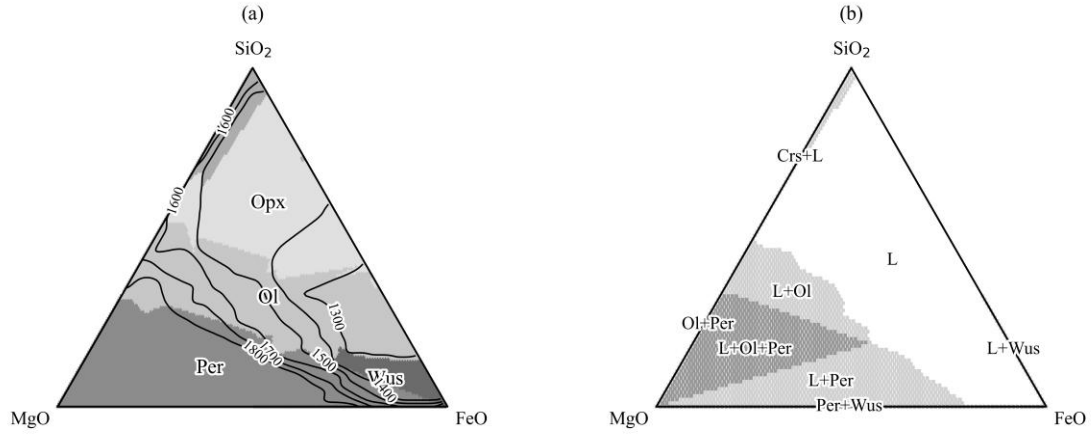

Figure 3: The assessed FeO-MgO-SiO_2 system calculated from the shipped database files. (a) Liquidus projection: primary crystallizing phase fields, with liquidus isotherms labeled in °C. (b) Isothermal section at 1600 °C: stable phase assemblages, shaded by the number of solid phases present. Phase labels: L liquid, Crs cristobalite, Ol olivine, Opx orthopyroxene, Per periclase, Wus wustite.

Table 2: The assessed liquids. Full source lists, modeling judgments, and limits are in each system's provenance file in the repository.

System	Fitted liquid excess (J/mol)	Fitted to	Reproduces; known limit
$CaO\text{-}SiO_2$	(-189764 + 15.71 T) + 57171 χ_{Ca}	silica activities [21,22]; bias-corrected melt-mixing enthalpies [28,29]; endmember and compound melting	mixing enthalpy near -58 kJ/mol at the metasilicate; silica-rich miscibility gap at the right composition but high in temperature
$MgO\text{-}SiO_2$	five terms in the equivalent fraction Y_{SiO2}, each a + b T	measured invariants and immiscibility data [25,26] from five studies, the measured consolute, and solution calorimetry [27]	all four invariants within about 40 °C; calorimetry matched; stable two-liquid field about 340 °C high
$FeO\text{-}SiO_2$	-42839 + 17.83 T	23 iron-saturated activities across two isotherms [23]	RMS ln a = 0.067; fayalite congruent melting within a few degrees
$FeO\text{-}MgO\text{-}SiO_2$	binaries combined; no ternary term	assembled from the binaries with olivine and orthopyroxene solutions [19,33]	measured liquidus-surface topology [24]; stable phase sets match pycalphad at 15 of 16 points

Anatomy of a database file

Because the database is half the contribution, it is worth stating plainly what such a file is. A ChemSage-format database is a self-contained plain-text file; the fitted binaries here are 41 to 59 lines, and the combined ternary with two solid solutions is 123. Those lines hold everything the calculation needs. The file opens with the elements and their masses. Each liquid carries its endmember Gibbs-energy functions as interval-wise polynomial coefficients, its cation charges and chemical groups, its quadruplet coordination numbers, and its excess terms with their composition exponents. Each solid solution lists its sublattice constituents, site ratios, and interaction terms, and each stoichiometric compound is a single line of coefficients. Nothing is opaque: every parameter that enters a calculated diagram can be read directly from the file.

The brevity of the file is inverted in the effort behind it. Building one from scratch, as done for each system here, starts with locating and triaging the primary literature, then extracting measurements with per-value provenance, including digitization from sixty-year-old figures. Endmember functions are assembled from evaluated compilations, the model structure is chosen, and the excess is fitted with the engine as the forward model. The result is validated against an independent implementation and against measurements withheld from the fit, and the limits are written down. The steps that resist automation are judgments: conflicting data sets must be arbitrated, reference states reconciled, and model-form limits recognized rather than papered

over with additional terms. dbbuild automates the mechanical part of this pipeline; the judgment does not compress.

That asymmetry is the economics of the field in miniature: implementations of the published models are increasingly available, but each fitted parameter condenses dozens of measurements and decisions. Parameter sets rather than codes are therefore the scarce commodity, and an openly licensed, provenance-tracked database is the part of this contribution that cannot be regenerated from the paper alone.

Quality control

Validation rests on one principle. A number is trusted only when an independent implementation reproduces it. pycalphad plays that role throughout, and Table 3 lists the checks. The pytest suite, 114 tests, compares every energy contribution on shared parameters and agrees to about 10^{-10} J per mole of atoms. The reader proves itself the hard way, by loading pycalphad's own test databases and reproducing their energies with no pycalphad at runtime. The compound energy formalism kernel matches pycalphad on a real multicomponent industrial file with charged species, vacancies, and logarithmic temperature terms. The multiphase construction is tested end to end: a combined file holding the MQMQA liquid, both solid solutions, and three stoichiometric oxides runs through pycalphad's own equilibrium calculation. The stable phase sets agree at 15 of 16 probe points, with Gibbs energies within about 10 J per mole of atoms; the one disagreement is a facet-resolution sliver at a field boundary. The WebAssembly build is validated in the browser against the same oracle values to machine precision. The TDB front-end is validated the same way, with pycalphad's own shipped test databases as the corpus: the subset-compatible alloy systems (Al-Zn, Pb-Sn, Al-Mg) load, and every solution phase matches pycalphad's Gibbs energy at random site fractions and temperatures to about 10^{-10} J per mole of atoms, line compounds included, while out-of-subset files must fail with the stated reason, and the suite asserts those failures too. The magnetic model is checked twice over: pycalphad parity on its own magnetic test database and on an own-transcribed Fe-C system, whose pure-iron transitions, carried almost entirely by the magnetic term, come out at 1185, 1668, and 1811 K against the measured 1184.8, 1667.5, and 1811. The Scheil panel is checked against known invariants: for Al-Zn at 30 mol% Zn it nucleates FCC_A1 at 836 K and freezes the 12.5% residue within one temperature step of the assessed 654 K eutectic [39]; for LiCl-KCl the residue freezes at 629 K against the measured 626 K.

Table 3: Validation against the independent oracle. Every check runs in the pytest suite except the in-browser one, which is exercised manually per release.

Check	Scope	Agreement
Energy paths (MQMQA reference, ideal, excess; CEF; stoichiometric)	shared parameter sets, every phase model	~10^{-10} J per mole of atoms
ChemSage reader	pycalphad's open test databases, no pycalphad at runtime	energies reproduced to machine precision
CEF kernel	multicomponent industrial file with charges, vacancies, ln T terms	machine precision
Multiphase equilibrium	liquid + two solid solutions + three oxides, 16 probe points	phase sets at 15 of 16; Gibbs energies within ~10 J per mole of atoms
WebAssembly build	in-browser against the same oracle values	machine precision
TDB reader	pycalphad's shipped alloy TDBs (Al-Zn, Pb-Sn, Al-Mg), every phase and line compound	~10^{-10} J per mole of atoms
TDB magnetic model	pycalphad parity + measured pure-iron transitions	matches to 10^{-10}; 1185/1668/1811 K
TDB scope guard	ionic-liquid and order-disorder test files	rejected, with the reason asserted
Gas phase	NASA-polynomial ideal-gas equilibria (Python and C core); Peng-Robinson real gas	Cantera [40] parity to 10^{-10} (Python), 10^{-6} (C/browser); real-gas compressibility to 10^{-3} vs Cantera PR

The database is tested against measurements rather than against other software. Endmember fusion points reproduce their sources exactly. The fitted FeO-SiO_2 liquid reproduces 23 measured activity points with a root-mean-square deviation of 0.067 in ln a [23], and the fitting route itself is regression-tested by re-deriving those published parameters from the raw data. Congruent melting of fayalite and forsterite lands within a few degrees of measurement, assessed MgO-SiO_2 invariants sit within about 40 °C, and the ternary liquidus projection reproduces the measured field topology [24]. Development practice is plain: the repository is version controlled with tagged releases (v0.1.0 through the v0.4.0 described here), the full suite runs before every release, and a fixed bug lands together with the regression test that would have caught it. The suite runs on Windows 11 under CPython 3.11 and 3.12; the browser application is exercised in Chromium- and Gecko-based browsers. A user can confirm a working installation by running pytest against the bundled test databases, or with no installation at all by opening the web application and loading the shipped ternary database.

(2) Availability

Operating system

Platform independent. The core builds with any C99 compiler; development and testing used Windows 11. The browser application runs in any current browser with WebAssembly support.

Programming language

C (core), Python 3.11 or later (interface and fitting tools), JavaScript (browser application).

Additional system requirements

None of note. The engine and database together are a few megabytes.

Dependencies

Python interface: cffi, NumPy, SciPy (developed and tested with cffi 2.0, NumPy 2.4, and SciPy 1.17). The validation suite additionally requires pycalphad; every oracle comparison reported here ran against pycalphad 0.11.1, and the tests that read pycalphad's own shipped test databases skip cleanly if a later version relocates them. The WebAssembly build requires emscripten. The C core and the browser application have no dependencies.

List of contributors

Michael E. Bustamante (design, implementation, validation, data curation).

Software location

Archive

- Name: GitHub Releases (release archive of the code repository)
- Persistent identifier: https://github.com/odinzen/hephaestus-mqmqa-public/releases/tag/v0.4.0
- Licence: MIT (code), CC BY 4.0 (database)
- Publisher: Michael E. Bustamante
- Version published: v0.4.0
- Date published: 2026-09-02

Code repository

- Name: GitHub
- Identifier: https://github.com/odinzen/hephaestus-mqmqa-public
- Licence: MIT (code), CC BY 4.0 (database, under data/)
- Date published: 2026-09-02

Language

English.

(3) Reuse potential

The nearest reuse is interoperation. Every database in the repository's data directory is a standard ChemSage file: pycalphad opens the same files this engine ships, files written by dbbuild load in both tools, and the test suite enforces that compatibility on every run, so a group already working in pycalphad can adopt the databases alone and ignore the engine. The reverse path holds too. A TDB an OpenCalphad or Thermo-Calc user already has loads in the browser unchanged, within the documented subset. A group that needs an embeddable solver can take the C core, two source directories of C99 with no dependencies, compiled by any C compiler. Its flat application binary interface (ABI) covers database reading, phase and parameter access, Gibbs evaluation, and liquid equilibration, callable from C, Python, or anything with a C foreign-function interface. The WebAssembly build is that same ABI passed through emscripten. Adopting uTDB is a reuse path of its own: the open specification together with the three shipped demonstration files and their round-trip suites amounts to a conformance kit for any implementation that wants to carry MQMQA inside TDB grammar.

The browser application makes the lowest barrier the default. A lecture on slag thermodynamics, a plant metallurgist screening a composition, or a reviewer checking a claimed equilibrium can load the database and calculate in seconds, with confidential inputs never leaving the machine. The same property makes it a practical template for other groups who want to publish a model as a zero-install tool.

Worked examples

Two complete sessions show the public API end to end. Both run against the shipped files as printed (the alloy session's ten-line lower_hull helper is printed in the repository copy), and Figures 4 and 5 are drawn from their output by plotting scripts kept in the repository. The salt session calculates the LiCl-KCl phase diagram:

```
import numpy as np

from mqmqa import Database
from mqmqa.equilibrium import build_inputs, multiphase_binary

db  = Database.read("web/LiCl-KCl.dat")         # the file the browser app ships
liq = db.phase_index("LICL-KCL-LIQUID")
LiCl, KCl = {"LI": 1, "CL": 1}, {"K": 1, "CL": 1}  # endmember compositions

diagram = []
for T in np.arange(500.0, 1101.0, 10.0):
    inp    = build_inputs(db, liq, T)         # liquid model at this temperature
    solids = [(0.0, db.stoich_gibbs(0, T), "LiCl(s)"),
              (1.0, db.stoich_gibbs(1, T), "KCl(s)")]
    for xi in np.arange(0.025, 1.0, 0.025):        # xi = mole fraction KCl
        state = multiphase_binary(inp, LiCl, KCl, solids, xi, ngrid=80)
        diagram.append((xi, T, "+".join(sorted(state["phases"]))))

liquid_only = [(T, xi) for xi, T, ph in diagram if ph == "LIQUID"]
Te, xe = min(liquid_only)                  # coldest all-liquid point = eutectic
```

In plain terms, the session sweeps temperature and composition steps, and at each point multiphase_binary compares the liquid solution against the two solid salts and reports which phases coexist. The two stoich_gibbs lines fetch the Gibbs energies of solid LiCl and solid KCl

at that temperature. The closing lines only search the results: among all points where the melt is entirely liquid, the coldest is the eutectic.

The listing prints its eutectic at x_{KCl} = 0.400 and 640 K at this deliberately coarse step size, against the measured 0.415 and 626 K; Figure 4 is the diagram it draws.

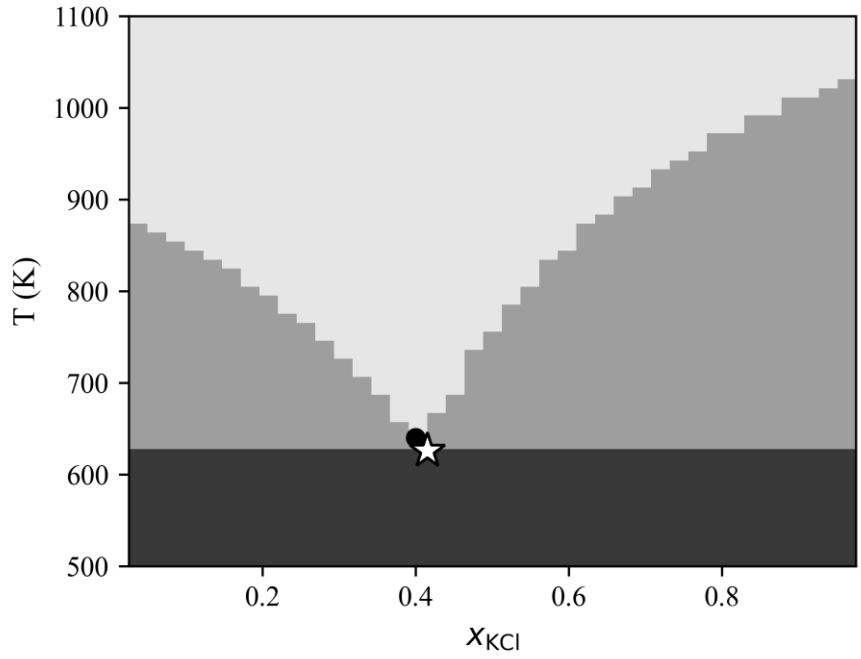

Figure 4: The LiCl-KCl phase diagram calculated by the salt listing from the shipped open database, at the listing's own step size. Light to dark: liquid, two-phase, and subsolidus fields. The dot is the calculated eutectic at this step size (0.400, 640 K); the star is the measured eutectic (0.415, 626 K).

The alloy session reads the Thermo-Calc dialect through the same API and calculates the Al-Zn diagram with a ten-line convex hull:

```
import numpy as np

from mqmqa import Database

db = Database.read("web/AlZn.tdb")            # Thermo-Calc dialect, same reader
phases = [(db.phase_index(n), n) for n in db.phase_names]

diagram = []
for T in np.arange(300.0, 1001.0, 5.0):
    pts = []
    for p, name in phases:                      # every CEF phase along the join
        for y in np.linspace(1e-4, 1-1e-4, 60):
            g = db.cef_gibbs(p, [1.0-y, y], T, per_mole_atoms=True)
            pts.append((y, g, name))
    hull = lower_hull(pts)                      # 2-D lower convex hull of (x, G)
    for edge_a, edge_b in zip(hull, hull[1:]):
        xa, _ga, name_a = edge_a
        xb, _gb, name_b = edge_b
        diagram.append((0.5*(xa+xb), T, tuple(sorted({name_a, name_b}))))
```

In plain terms: at each temperature the loop evaluates every solid-solution phase's Gibbs energy across the whole composition line (cef_gibbs), and lower_hull keeps only the states no mixture of others can beat, the geometric equivalent of the common-tangent construction. Where two neighboring hull points belong to different phases, those phases coexist.

It locates the eutectic at x_{Zn} = 0.890 and 660 K against the assessed 0.885 and 654 K [39] (Figure 5). The shipped AlZn.tdb is itself an openly licensed transcription of that published assessment on the Scientific Group Thermodata Europe (SGTE) unary functions [41], verified bit-identical through pycalphad against the file it transcribes. The slag systems run through the same calls behind Figures 2 and 3, and the repository's validation scripts are their executable record.

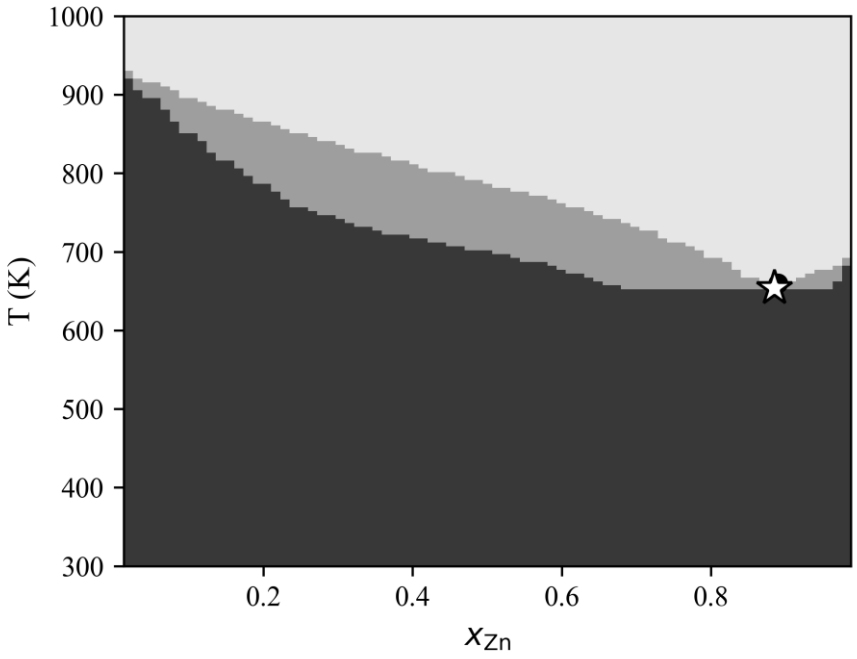

Figure 5: The Al-Zn phase diagram calculated by the alloy listing from the shipped open TDB. Light to dark: liquid, liquid plus solid, and solid fields. The dot is the calculated eutectic (0.890, 660 K); the star is the assessed eutectic (0.885, 654 K).

The database is a starting point rather than an endpoint, and the file format rewards growth: new components, new phases, and refits against new measurements accumulate in the same database. A system is therefore extended release by release rather than rebuilt, and archived versions keep results calculated against earlier releases reproducible. Because every value carries its source and every judgment is written down, another assessor can reweight the data, swap an endmember, or extend a system without reverse-engineering anything; dbbuild turns a table of measured activities into a loadable file in a few lines. The in-browser multiphase hull already covers any loaded three-cation file and any loaded alloy TDB join, so a new system becomes a browser phase diagram the moment its file loads. Natural extensions are further oxide systems, the remaining ChemSage solution models, the ionic-liquid TDB model, and multicomponent Scheil on the same hull machinery. The databases carry per-value provenance but not parameter uncertainties; refitting the same measured-data tables through ESPEI [10] to obtain Bayesian uncertainty bounds is a natural continuation. A first reach into the combustion school is already

in the library. The engine reads NASA 7-coefficient polynomial thermochemistry, the open format of NASA CEA [42] and GRI-Mech [43], and solves ideal-gas equilibrium at fixed temperature and pressure by the element-potential method, validated against Cantera [40] to 10^{-10} mole fraction from 1200 to 3000 K and 0.1 to 10 atm. The same solver is compiled into the WebAssembly core, so it runs in the browser gas calculator alongside the condensed engine. For the dense conditions of pressurized reactors and gasifiers, a Peng-Robinson real-gas option applies fugacity corrections from tabulated critical constants, validated against Cantera's own Peng-Robinson and reducing to the ideal result at atmospheric pressure. Coupling the gas to a molten oxide, for pyrometallurgical oxygen potentials, is the step still in progress. The uTDB grammar already defines the ideal-gas phase, so a single file can in principle carry an alloy, a slag, its solids, and a gas. The repository also publishes an openly specified unified dialect (uTDB) that carries the MQMQA statements inside TDB grammar, with two shipped demonstration files chosen to be processes rather than curiosities: an aluminum-recycling database holding the Al-Zn alloy with its NaCl-KCl-MgCl2 salt flux, and a steelmaking database holding a basic Fe-C steel, magnetic phases included, with the FeO-MgO-SiO2 slag and its silicate solid solutions. Round-trip tests assert both reproduce their source-file physics at machine precision. The steelmaking file is the clearest illustration of what a unified container buys (Figure 6): from one file the browser calculates the iron-carbon alloy diagram through the magnetic compound energy formalism and its Scheil solidification path, while the same file's slag is a molten-salt-model liquid, two physics that no single legacy container holds together. Relative to the XML road of XTDB [15] (Table 1), uTDB is the on-ramp from the installed base, extending files as they are and running in this engine today, with mechanical conversion to XTDB elements the natural bridge once that schema settles. Making the calculator multilingual across the field's file dialects is the direction of travel. Contributions and issues are handled through the GitHub repository.

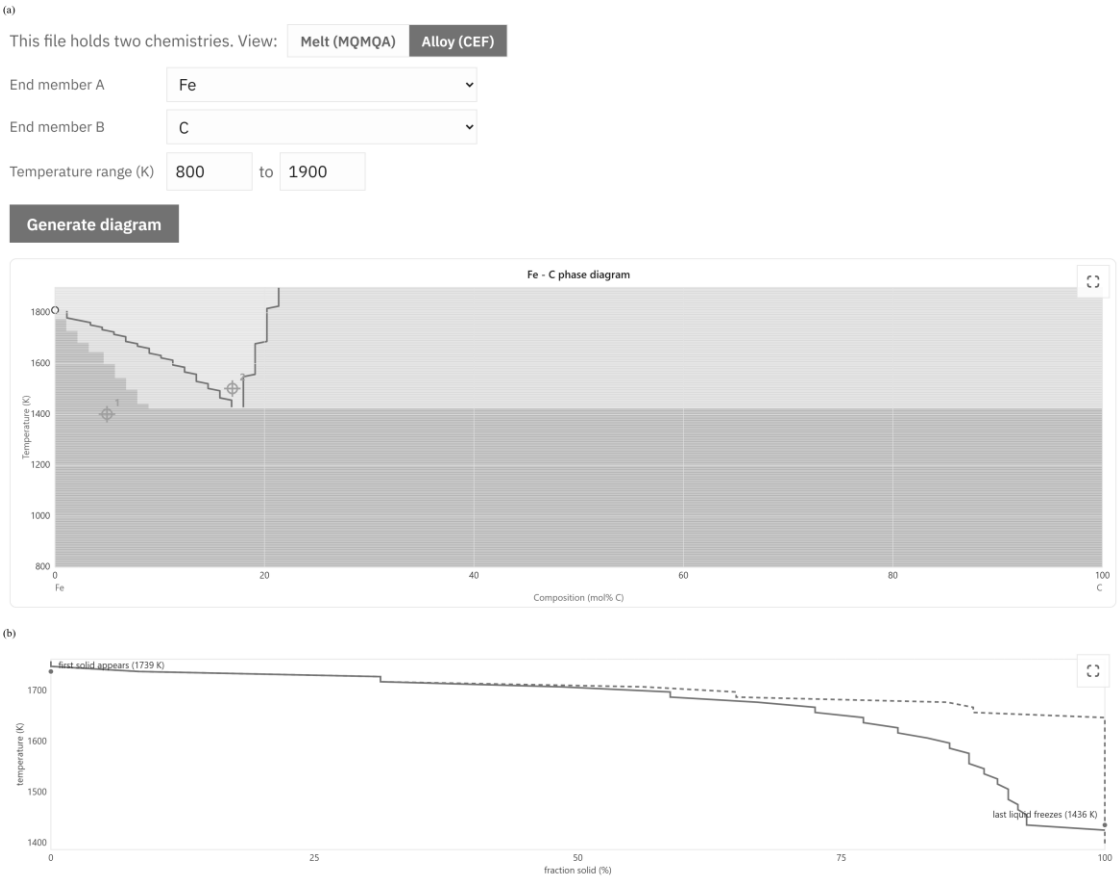

Figure 6: One unified (.utdb) file, two physics, calculated in the browser. (a) The steelmaking database loaded with the Melt/Alloy view toggle on Alloy; the engine calculates the Fe-C phase diagram through the magnetic compound energy formalism, with two tapped compositions pinned. (b) Scheil solidification for the same join at 4 mol% C, first solid at 1739 K and the residual liquid frozen by 1436 K, against the equilibrium lever-rule path (dashed). The file's slag half is a Modified Quasichemical Model liquid; no legacy container holds both models at once.

The gap this work addresses was never the model, which has been published for two decades, but an open, runnable pairing of model and parameters. Hephaestus supplies both halves under open licenses and puts them behind a browser page that asks nothing of its user. The claims are checkable end to end. Release v0.4.0 archives the exact version described, and the 114-test suite reruns the pycalphad comparisons on any machine. The worked examples print the figures shown, and every parameter in every shipped database traces to a published measurement through its provenance file. A reader who doubts a number can recalculate it; one who needs a system that is missing can build it with the same open tools, or ask.

Data availability

The engine, the database, the browser application, and the validation suite are openly available in the code repository listed above; the version described here is archived as release v0.4.0. The database files and their provenance records are in the repository's data directory under CC BY 4.0.

Acknowledgements

During the development of this software the author used Claude Code (Anthropic) to assist with implementation, validation scripting, and manuscript preparation. This assistance was agentic, not autonomous: it operated under author direction and supervision at every step and was never a closed loop that produced or accepted results without review. Every parameter was traced by the author to its primary source before entering the database, and every reference was verified against its registry record. The author reviewed and approved all output and takes full responsibility for this article.

Funding statement

The development of this software received no external funding.

Competing interests

The author founded Odinzen LLC, which provides commercial thermodynamic modeling services, and declares this as a competing interest. The software and database published here are released in full under open licenses, and no proprietary Odinzen assets are included.

References

[1] Pelton AD, Chartrand P, Eriksson G. The modified quasi-chemical model: Part IV. Two-sublattice quadruplet approximation. Metall Mater Trans A. 2001 Jun;32(6):1409–16. doi:10.1007/s11661-001-0230-7

[2] Pelton AD, Degterov SA, Eriksson G, Robelin C, Dessureault Y. The modified quasichemical model I—Binary solutions. Metall Mater Trans B. 2000 Aug;31(4):651–9. doi:10.1007/s11663-000-0103-2

[3] Pelton AD, Blander M. Thermodynamic analysis of ordered liquid solutions by a modified quasichemical approach—Application to silicate slags. Metall Trans B. 1986 Dec;17(4):805–15. doi:10.1007/BF02657144

[4] Bale CW, Bélisle E, Chartrand P, Decterov SA, Eriksson G, Gheribi AE, et al. FactSage thermochemical software and databases, 2010–2016. Calphad. 2016 Sep;54:35–53. doi:10.1016/j.calphad.2016.05.002

[5] Jung IH. Overview of the applications of thermodynamic databases to steelmaking processes. Calphad. 2010 Sep;34(3):332–62. doi:10.1016/j.calphad.2010.06.003

[6] Jung IH, Van Ende MA. Computational Thermodynamic Calculations: FactSage from CALPHAD Thermodynamic Database to Virtual Process Simulation. Metall Mater Trans B. 2020 Oct;51(5):1851–74. doi:10.1007/s11663-020-01908-7

[7] Sundman B, Kattner UR, Palumbo M, Fries SG. OpenCalphad - a free thermodynamic software. Integr Mater Manuf Innov. 2015 Dec;4(1):1–15. doi:10.1186/s40192-014-0029-1

[8] Otis R, Liu ZK. pycalphad: CALPHAD-based Computational Thermodynamics in Python. JORS. 2017 Jan 9;5(1):1. doi:10.5334/jors.140

[9] Paz Soldan Palma J, Gong R, Bocklund BJ, Otis R, Poschmann M, Piro M, et al. Thermodynamic modeling with uncertainty quantification using the modified quasichemical model in quadruplet approximation: Implementation into PyCalphad and ESPEI. Calphad. 2023 Dec;83:102618. doi:10.1016/j.calphad.2023.102618

[10] Bocklund B, Otis R, Egorov A, Obaied A, Roslyakova I, Liu ZK. ESPEI for efficient thermodynamic database development, modification, and uncertainty quantification: application to Cu–Mg. MRS Communications. 2019 Jun;9(2):618–27. doi:10.1557/mrc.2019.59

[11] Poschmann M, Bajpai P, Fitzpatrick BWN, Piro MHA. Recent developments for molten salt systems in Thermochimica. Calphad. 2021 Dec;75:102341. doi:10.1016/j.calphad.2021.102341

[12] Campbell CE, Kattner UR, Liu ZK. The development of phase-based property data using the CALPHAD method and infrastructure needs. Integr Mater Manuf Innov. 2014 Dec;3(1):158–80. doi:10.1186/2193-9772-3-12

[13] Andersson JO, Helander T, Höglund L, Shi P, Sundman B. Thermo-Calc & DICTRA, computational tools for materials science. Calphad. 2002 Jun;26(2):273-312. doi:10.1016/S0364-5916(02)00037-8

[14] Kulik DA, Winnefeld F, Kulik A, Miron GD, Lothenbach B. CemGEMS - an easy-to-use web application for thermodynamic modeling of cementitious materials. RILEM Tech Lett. 2021;6:36-52. doi:10.21809/rilemtechlett.2021.140

[15] Sundman B, Miani F, van de Walle A, Hallstedt B, Kattner UR, Tang F, et al. XTDB, an XML based format for Calphad databases. Calphad. 2025 Sep;90:102849. doi:10.1016/j.calphad.2025.102849

[16] Jain A, Ong SP, Hautier G, Chen W, Richards WD, Dacek S, et al. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. APL Mater. 2013;1(1):011002. doi:10.1063/1.4812323

[17] Hillert M. The compound energy formalism. Journal of Alloys and Compounds. 2001 May;320(2):161–76. doi:10.1016/S0925-8388(00)01481-X

[18] Eriksson G, Hack K. ChemSage - A computer program for the calculation of complex chemical equilibria. Metall Trans B. 1990 Dec;21(6):1013-23. doi:10.1007/BF02670272

[19] Robie RA, Hemingway BS. Thermodynamic properties of minerals and related substances at 298.15 K and 1 bar (10^5 pascals) pressure and at higher temperatures. U.S. Geological Survey Bulletin 2131. Washington: U.S. Government Printing Office; 1995.

[20] Chase MW Jr. NIST-JANAF Thermochemical Tables, 4th edition. Journal of Physical and Chemical Reference Data, Monograph 9. Woodbury, NY: American Institute of Physics; 1998.

[21] Kay DAR, Taylor J. Activities of silica in the lime + alumina + silica system. Trans Faraday Soc. 1960;56:1372. doi:10.1039/tf9605601372

[22] Stolyarova VL, Shornikov SI, Ivanov GG, Shultz MM. High Temperature Mass Spectrometric Study of Thermodynamic Properties of the CaO-SiO2 System. J Electrochem Soc. 1991 Dec 1;138(12):3710–4. doi:10.1149/1.2085485

[23] Björkman B. An assessment of the system Fe-O-SiO2 using a structure based model for the liquid silicate. Calphad. 1985 Jul;9(3):271–82. doi:10.1016/0364-5916(85)90012-4

[24] Bowen NL, Schairer JF. The system MgO-FeO-SiO2. American Journal of Science. 1935 Feb 1;s5-29(170):151–217. doi:10.2475/ajs.s5-29.170.151

[25] Greig JW. Immiscibility in silicate melts; Part I. American Journal of Science. 1927 Jan 1;s5-13(73):1–44. doi:10.2475/ajs.s5-13.73.1

[26] Greig JW. Immiscibility in silicate melts; Part II. American Journal of Science. 1927 Feb 1;s5-13(74):133–54. doi:10.2475/ajs.s5-13.74.133

[27] Charlu TV, Newton RC, Kleppa OJ. Enthalpies of formation at 970 K of compounds in the system MgO-Al2O3-SiO2 from high temperature solution calorimetry. Geochimica et Cosmochimica Acta. 1975 Nov;39(11):1487–97. doi:10.1016/0016-7037(75)90150-7

[28] Yang H, Hu C, Zhou Y, Liu X, Shi Y, Li J, et al. MatterSim: A Deep Learning Atomistic Model Across Elements, Temperatures and Pressures. arXiv; 2024. doi:10.48550/ARXIV.2405.04967

[29] Neumann M, Gin J, Rhodes B, Bennett S, Li Z, Choubisa H, et al. Orb: A Fast, Scalable Neural Network Potential. arXiv; 2024. doi:10.48550/ARXIV.2410.22570

[30] Giannozzi P, Baroni S, Bonini N, Calandra M, Car R, Cavazzoni C, et al. QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials. J Phys Condens Matter. 2009;21(39):395502. doi:10.1088/0953-8984/21/39/395502

[31] Kresse G, Furthmüller J. Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set. Phys Rev B. 1996 Oct;54(16):11169-86. doi:10.1103/PhysRevB.54.11169

[32] Park Y, Kim J, Hwang S, Han S. Scalable Parallel Algorithm for Graph Neural Network Interatomic Potentials in Molecular Dynamics Simulations. J Chem Theory Comput. 2024;20(13):4857-68. doi:10.1021/acs.jctc.4c00190

[33] Wood BJ, Kleppa OJ. Thermochemistry of forsterite-fayalite olivine solutions. Geochimica et Cosmochimica Acta. 1981 Apr;45(4):529–34. doi:10.1016/0016-7037(81)90185-X

[34] Wu P, Eriksson G, Pelton AD, Blander M. Prediction of the Thermodynamic Properties and Phase Diagrams of Silicate Systems-Evaluation of the FeO-MgO-SiO2 System. ISIJ International. 1993;33(1):26–35. doi:10.2355/isijinternational.33.26

[35] Barin I. Thermochemical Data of Pure Substances. 3rd ed. Weinheim: VCH; 1995. doi:10.1002/9783527619825

[36] Hersh LS, Kleppa OJ. Enthalpies of Mixing in Some Binary Liquid Halide Mixtures. J Chem Phys. 1965;42:1309. doi:10.1063/1.1696115

[37] Sangster J, Pelton AD. Phase Diagrams and Thermodynamic Properties of the 70 Binary Alkali Halide Systems Having Common Ions. J Phys Chem Ref Data. 1987;16:509. doi:10.1063/1.555803

[38] Ard JC, Yingling JA, Johnson TJ, Schorne-Pinto J, Aziziha M, Dixon CM, et al. Development of the Molten Salt Thermal Properties Database - Thermochemical (MSTDB-TC), example applications, and LiCl-RbCl and UF3-UF4 system assessments. J Nucl Mater. 2022 May;563:153631. doi:10.1016/j.jnucmat.2022.153631

[39] an Mey S. Reevaluation of the Al-Zn system. Z Metallkd. 1993;84(7):451-5. doi:10.1515/ijmr-1993-840704

[40] Goodwin DG, Moffat HK, Schoegl I, Speth RL, Weber BW. Cantera: An Object-oriented Software Toolkit for Chemical Kinetics, Thermodynamics, and Transport Processes. Version 3.1.0; 2024. doi:10.5281/zenodo.14455267

[41] Dinsdale AT. SGTE data for pure elements. Calphad. 1991 Oct;15(4):317-425. doi:10.1016/0364-5916(91)90030-N

[42] McBride BJ, Zehe MJ, Gordon S. NASA Glenn Coefficients for Calculating Thermodynamic Properties of Individual Species. NASA/TP-2002-211556. Cleveland (OH): NASA Glenn Research Center; 2002.

[43] Smith GP, Golden DM, Frenklach M, Moriarty NW, Eiteneer B, Goldenberg M, et al. GRI-Mech 3.0 [Internet]. 1999. Available from: http://combustion.berkeley.edu/gri-mech/